

EXPERIMENT NO. 9

OSSP - Operating Systems and Systems Programming

Course Code: 25CS2104E

Task 1

File Copy Using Low-Level System Calls (open, read, write, lseek, close)

```
/*
 * Experiment 9 - Task 1: File Copy using Low-Level POSIX System Calls
 *
 * System Calls Demonstrated:
 * - open(): Opens source and destination files and returns file descriptors.
 * - read(): Reads a block of bytes from the source file into a buffer.
 * - write(): Writes bytes from buffer into destination file.
 * - lseek(): Demonstrates finding file size and repositioning file offset.
 * - close(): Closes file descriptors to release system resources.
 */

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>

#define BUFFER_SIZE 64

int main(int argc, char *argv[]) {
    char *src_filename = "source.txt";
    char *dest_filename = "destination.txt";

    if (argc >= 3) {
        src_filename = argv[1];
        dest_filename = argv[2];
    }

    printf("=== POSIX System Call File Copy Utility ===\n");
    printf("Source file: %s\n", src_filename);
    printf("Destination file: %s\n", dest_filename);

    // 1. open() source file in read-only mode
    int src_fd = open(src_filename, O_RDONLY);
    if (src_fd < 0) {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }
    printf("[1] open(): Source file opened successfully (fd = %d)\n", src_fd);

    // 2. lseek() demonstration: Find total size and rewind offset
    off_t file_size = lseek(src_fd, 0, SEEK_END);
    if (file_size < 0) {
        perror("Error determining file size with lseek");
        close(src_fd);
        exit(EXIT_FAILURE);
    }
    printf("[2] lseek(): Measured source file size = %lld bytes\n", (long long)file_size);

    // Rewind back to the start of the file for copying
    if (lseek(src_fd, 0, SEEK_SET) < 0) {
        perror("Error rewinding file with lseek");
        close(src_fd);
        exit(EXIT_FAILURE);
    }
    printf("[3] lseek(): File offset reset to 0 (beginning of file)\n");

    // 3. open() destination file in write-only mode, create if missing, truncate if exists
    // Permissions: 0644 (rw-r--r--)
    int dest_fd = open(dest_filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd < 0) {
        perror("Error opening destination file");
        close(src_fd);
        exit(EXIT_FAILURE);
    }
}
```

```

printf("[4] open(): Destination file opened successfully (fd = %d)\n", dest_fd);

// 4. read() and write() loop
char buffer[BUFFER_SIZE];
ssize_t bytes_read, bytes_written;
ssize_t total_copied = 0;

printf("[5] Copying data in chunks of %d bytes...\n", BUFFER_SIZE);
while ((bytes_read = read(src_fd, buffer, sizeof(buffer))) > 0) {
    bytes_written = write(dest_fd, buffer, bytes_read);
    if (bytes_written != bytes_read) {
        perror("Error writing to destination file");
        close(src_fd);
        close(dest_fd);
        exit(EXIT_FAILURE);
    }
    total_copied += bytes_written;
}

if (bytes_read < 0) {
    perror("Error reading from source file");
    close(src_fd);
    close(dest_fd);
    exit(EXIT_FAILURE);
}

// 5. close() both file descriptors
close(src_fd);
close(dest_fd);
printf("[6] close(): Both file descriptors closed successfully\n");

printf("\nSuccess! Copied %zd bytes from %s to %s.\n", total_copied, src_filename, dest_filename);
return 0;
}

```

Output:

```

zsh - copy_syscall

rishivarma@MacBook-Air ~ % ./copy_syscall source.txt destination.txt
=== POSIX System Call File Copy Utility ===
Source file: source.txt
Destination file: destination.txt
[1] open(): Source file opened successfully (fd = 3)
[2] lseek(): Measured source file size = 216 bytes
[3] lseek(): File offset reset to 0 (beginning of file)
[4] open(): Destination file opened successfully (fd = 5)
[5] Copying data in chunks of 64 bytes...
[6] close(): Both file descriptors closed successfully
Success! Copied 216 bytes from source.txt to destination.txt.
=== DESTINATION FILE CONTENT ===
Operating Systems and Systems Programming (OSSP) Lab.
Experiment 9 Task 1: Low-level file copying using POSIX system calls.
System calls demonstrated: open, read, write, lseek, close.
Hello from Linux/UNIX file I/O!
rishivarma@MacBook-Air ~ %

```

Task 2

File Copy Using Standard C Library Functions (fopen, fread, fwrite, fclose)

```

/*
 * Experiment 9 - Task 2: File Copy using Standard C Library Functions
 *
 * Standard Library Functions Demonstrated:
 * - fopen(): Opens files in binary/text mode and returns buffered FILE* streams.
 * - fread(): Reads blocks of objects/bytes from the input stream.
 * - fwrite(): Writes blocks of objects/bytes to the output stream.
 * - fclose(): Flushes buffers and closes the streams.
 */

```

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 64

int main(int argc, char *argv[]) {
    char *src_filename = "source.txt";
    char *dest_filename = "destination.txt";

    if (argc >= 3) {
        src_filename = argv[1];
        dest_filename = argv[2];
    }

    printf("=== Standard C Library File Copy Utility ===\n");
    printf("Source file: %s\n", src_filename);
    printf("Destination file: %s\n", dest_filename);

    // 1. fopen() source file in read mode ("rb" for binary/text safety)
    FILE *src_fp = fopen(src_filename, "rb");
    if (src_fp == NULL) {
        perror("Error opening source file with fopen");
        exit(EXIT_FAILURE);
    }
    printf("[1] fopen(): Source file opened successfully (FILE* stream created)\n");

    // 2. fopen() destination file in write mode ("wb")
    FILE *dest_fp = fopen(dest_filename, "wb");
    if (dest_fp == NULL) {
        perror("Error opening destination file with fopen");
        fclose(src_fp);
        exit(EXIT_FAILURE);
    }
    printf("[2] fopen(): Destination file opened successfully\n");

    // 3. fread() and fwrite() loop
    char buffer[BUFFER_SIZE];
    size_t bytes_read, bytes_written;
    size_t total_copied = 0;

    printf("[3] Copying buffered data in chunks of %d bytes...\n", BUFFER_SIZE);
    while ((bytes_read = fread(buffer, 1, sizeof(buffer), src_fp)) > 0) {
        bytes_written = fwrite(buffer, 1, bytes_read, dest_fp);
        if (bytes_written != bytes_read) {
            perror("Error writing data with fwrite");
            fclose(src_fp);
            fclose(dest_fp);
            exit(EXIT_FAILURE);
        }
        total_copied += bytes_written;
    }

    if (ferror(src_fp)) {
        perror("Error reading data with fread");
        fclose(src_fp);
        fclose(dest_fp);
        exit(EXIT_FAILURE);
    }

    // 4. fclose() both streams
    fclose(src_fp);
    fclose(dest_fp);
    printf("[4] fclose(): Both streams flushed and closed successfully\n");

    printf("\nSuccess! Copied %zu bytes from %s to %s.\n", total_copied, src_filename, dest_filename);
    return 0;
}
```

Output:

```

zsh - copy_stdio

rishivarma@MacBook-Air ~ % ./copy_stdio source.txt destination.txt
=== Standard C Library File Copy Utility ===
Source file: source.txt
Destination file: destination.txt
[1] fopen(): Source file opened successfully (FILE* stream created)
[2] fopen(): Destination file opened successfully
[3] Copying buffered data in chunks of 64 bytes...
[4] fclose(): Both streams flushed and closed successfully
Success! Copied 226 bytes from source.txt to destination.txt.
=== DESTINATION FILE CONTENT ===
Operating Systems and Systems Programming (OSSP) Lab.
Experiment 9 Task 2: Standard C Library File Copying.
Functions demonstrated: fopen, fread, fwrite, fclose.
Standard library uses user-space buffered I/O streams (FILE *).
rishivarma@MacBook-Air ~ %

```

Task 3**File Offset Manipulation Using lseek()**

```

/*
 * Experiment 9 - Task 3: Demonstration of lseek() System Call
 *
 * Demonstrates moving the read/write file offset:
 * - SEEK_SET: Offset is set relative to the beginning of the file.
 * - SEEK_CUR: Offset is set relative to the current file position.
 * - SEEK_END: Offset is set relative to the end of the file.
 */

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    char *filename = "sample.txt";
    if (argc >= 2) {
        filename = argv[1];
    }

    printf("=== lseek() System Call Demonstration ===\n");
    printf("Opening file: %s\n\n", filename);

    int fd = open(filename, O_RDONLY);
    if (fd < 0) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }

    char buf[64];
    ssize_t bytes_read;
    off_t current_pos;

    // 1. Initial read from position 0
    bytes_read = read(fd, buf, 10);
    if (bytes_read > 0) {
        buf[bytes_read] = '\0';
        current_pos = lseek(fd, 0, SEEK_CUR);
        printf("[Step 1] Read 10 bytes from beginning:\n");
        printf("    Data: \"%s\"\n", buf);
        printf("    Current offset position: %lld\n\n", (long long)current_pos);
    }

    // 2. lseek with SEEK_SET: jump to offset 15 from beginning
    off_t new_pos = lseek(fd, 15, SEEK_SET);
    printf("[Step 2] lseek(fd, 15, SEEK_SET):\n");
    printf("    Offset moved to: %lld\n", (long long)new_pos);
}

```

```

bytes_read = read(fd, buf, 10);
if (bytes_read > 0) {
    buf[bytes_read] = '\0';
    printf("        Read 10 bytes: \"%s\"\n\n", buf);
}

// 3. lseek with SEEK_CUR: skip forward 5 bytes from current position
new_pos = lseek(fd, 5, SEEK_CUR);
printf("[Step 3] lseek(fd, 5, SEEK_CUR):\n");
printf("        Offset jumped ahead 5 bytes to: %lld\n", (long long)new_pos);
bytes_read = read(fd, buf, 10);
if (bytes_read > 0) {
    buf[bytes_read] = '\0';
    printf("        Read 10 bytes: \"%s\"\n\n", buf);
}

// 4. lseek with SEEK_END: reposition to 15 bytes before end of file
new_pos = lseek(fd, -15, SEEK_END);
printf("[Step 4] lseek(fd, -15, SEEK_END):\n");
printf("        Offset moved to (EOF - 15 bytes): %lld\n", (long long)new_pos);
bytes_read = read(fd, buf, 14);
if (bytes_read > 0) {
    buf[bytes_read] = '\0';
    printf("        Read 14 bytes: \"%s\"\n\n", buf);
}

close(fd);
printf("File closed successfully.\n");
return 0;
}

```

Output:

```

zsh - lseek_demo

rishivarma@MacBook-Air ~ % ./lseek_demo sample.txt
=== lseek() System Call Demonstration ===
Opening file: sample.txt
[Step 1] Read 10 bytes from beginning:
Data: "ABCDEFGHIIJ"
Current offset position: 10
[Step 2] lseek(fd, 15, SEEK_SET):
Offset moved to: 15
Read 10 bytes: "PQRSTUVWXYZ"
[Step 3] lseek(fd, 5, SEEK_CUR):
Offset jumped ahead 5 bytes to: 30
Read 10 bytes: "3456789 - "
[Step 4] lseek(fd, -15, SEEK_END):
Offset moved to (EOF - 15 bytes): 84
Read 14 bytes: "SSP) Lab Demo!"
File closed successfully.
rishivarma@MacBook-Air ~ %

```

Task 4

Standard Output Redirection Using File Descriptors and dup2()

```

/*
 * Experiment 9 - Task 4: Output Redirection using open(), dup2(), and close()
 *
 * Demonstrates shell-style output redirection (e.g., ./program > file.txt):
 * 1. open(): Opens or creates the target redirect file.
 * 2. dup2(fd, STDOUT_FILENO): Duplicates the file descriptor onto STDOUT_FILENO (fd 1)
 *
 * 3. close(fd): Closes the duplicate file descriptor.
 * 4. printf(): All subsequent standard output is redirected into the target file!
 * 5. Restores original STDOUT using a saved descriptor to show how shell manages I/O.
 */

#include <stdio.h>
#include <stdlib.h>

```

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    const char *redirect_filename = "redirected_output.txt";

    printf("=== Demonstration of Output Redirection Using dup2() ===\n");
    printf("[Step 1] This message is printed to the original TERMINAL (stdout).\n");
    printf("[Step 2] Saving original stdout descriptor using dup(STDOUT_FILENO)...\n");

    // Save current STDOUT_FILENO so we can restore terminal printing later
    int saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout < 0) {
        perror("Error saving stdout with dup");
        exit(EXIT_FAILURE);
    }

    // Open target redirection file
    int file_fd = open(redirect_filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file_fd < 0) {
        perror("Error opening redirect file");
        close(saved_stdout);
        exit(EXIT_FAILURE);
    }
    printf("[Step 3] Opened '%s' with file descriptor %d.\n", redirect_filename, file_fd);
    printf("[Step 4] Redirecting STDOUT_FILENO to '%s' using dup2()...\n", redirect_filename);
    fflush(stdout); // Flush existing stdout before redirecting descriptor

    // Redirect standard output (fd 1) to point to the file
    if (dup2(file_fd, STDOUT_FILENO) < 0) {
        perror("Error redirecting stdout with dup2");
        close(file_fd);
        close(saved_stdout);
        exit(EXIT_FAILURE);
    }

    // Close the original file descriptor; STDOUT_FILENO now references the file
    close(file_fd);

    // These printf statements will NOT appear on the terminal! They write directly to the file!
    printf("=====\n");
    printf("HELLO FROM REDIRECTED STDOUT!\n");
    printf("This output was generated using standard printf().\n");
    printf("Because dup2 redirected STDOUT_FILENO (fd 1) to '%s',\n", redirect_filename);
    ;
    printf("this text was captured into the file instead of the terminal.\n");
    printf("This is the exact mechanism Unix shells use for '>' redirection.\n");
    printf("=====\n");

    // Flush user-space buffer before switching file descriptors back
    fflush(stdout);

    // Restore original terminal stdout
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("[Step 5] STDOUT_FILENO has been successfully RESTORED back to the terminal!\n");
    printf("[Step 6] Redirection test complete. Inspect '%s' to verify.\n", redirect_filename);

    return 0;
}
```

Output:

```
zsh - redirect_demo

rishivarma@MacBook-Air ~ % ./redirect_demo && cat redirected_output.txt
=== Demonstration of Output Redirection Using dup2() ===
[Step 1] This message is printed to the original TERMINAL (stdout).
[Step 2] Saving original stdout descriptor using dup(STDOUT_FILENO)...
[Step 3] Opened 'redirected_output.txt' with file descriptor 5.
[Step 4] Redirecting STDOUT_FILENO to 'redirected_output.txt' using dup2()...
[Step 5] STDOUT_FILENO has been successfully RESTORED back to the terminal!
[Step 6] Redirection test complete. Inspect 'redirected_output.txt' to verify.
=== CONTENTS OF redirected_output.txt ===
=====
HELLO FROM REDIRECTED STDOUT!
This output was generated using standard printf().
Because dup2 redirected STDOUT_FILENO (fd 1) to 'redirected_output.txt',
this text was captured into the file instead of the terminal.
This is the exact mechanism Unix shells use for '>' redirection.
=====
rishivarma@MacBook-Air ~ %
```